

Real-Time Fish Swarm Simulation for Minecraft

Suggested architecture: Boids + voxel-aware obstacle avoidance + school-level behavior

Executive summary

For a Minecraft mod, the recommended approach is a hybrid steering architecture rather than conventional per-fish pathfinding. Use Boids for local social behavior, Minecraft's voxel/block representation for obstacle sensing, and a higher-level school controller for goals such as wandering, feeding, fleeing predators, and migration. This keeps behavior organic while taking advantage of the fact that Minecraft already exposes a discrete 3D environment.

1. High-level architecture

The simulation should be organized into four layers: world/environment sensing, school-level behavior, individual flocking, and movement/animation.

Layer	Responsibility	Typical update
World / voxel sensing	Water/solid classification, clearance, local obstacle probes	Near fish: frequent
School controller	Target selection, wandering, feeding, fleeing, migration, school direction	Lower frequency
Individual fish	Separation, alignment, cohesion, local avoidance, variation	Near fish: 20–60 Hz
Movement / animation	Acceleration, orientation, swim cycle, body bend	Render/update loop

2. Individual fish steering

Each fish computes a desired steering vector from several weighted components. The exact weights should be tuned empirically rather than treated as fixed constants.

```
desiredSteering = separation × Ws + alignment × Wa + cohesion × Wc + schoolDirection × Wschool +  
obstacleAvoidance × Wavoid + environment × Wenv
```

Separation prevents crowding; alignment produces coordinated headings; cohesion keeps the group together; school direction provides large-scale intent; obstacle avoidance handles local geometry; environmental steering can incorporate currents or other world effects.

3. Neighbor lookup

Do not compare every fish against every other fish. Maintain a spatial hash or uniform 3D grid keyed by block/cell coordinates. A fish then inspects only nearby cells within its interaction radius. This changes the practical workload from an $O(N^2)$ all-pairs search toward $O(N \cdot k)$, where k is the local neighbor count.

For Minecraft, a uniform grid is especially attractive because world coordinates are already discrete and chunk/block locality is readily available.

4. Voxel-aware obstacle avoidance

The preferred local obstacle system is a voxel-aware sensor rather than conventional navigation for every fish. Sample a short 3D volume around and ahead of each fish. Classify sampled blocks as traversable water, open space, solid obstacles, or special/custom blocks.

Use a fan of candidate directions or probes instead of a single forward ray. Score each candidate by alignment with the desired direction, school direction, and available clearance. Select the best direction and smoothly steer toward it.

Conceptually: $\text{score}(\text{direction}) = \text{desiredDirection} \cdot \text{direction} \times W_{\text{goal}} + \text{schoolDirection} \cdot \text{direction} \times W_{\text{school}} + \text{clearance} \times W_{\text{clearance}} - \text{obstacleCost} \times W_{\text{obstacle}}$

This works well for Minecraft's one-block pillars, cave walls, overhangs, underwater structures, and irregular voxel geometry.

5. Temporary escape targets

When an obstacle is detected, create a short-lived avoidance target in a clear direction rather than allowing raw avoidance forces to change every frame. The fish steers toward that target until its local clearance is restored, then returns control to flocking.

This stateful behavior helps prevent wall jitter and repeated left-right oscillation.

6. School-level controller

A school should have its own lightweight state: center, preferred direction, target, speed range, behavioral mode, panic level, and perhaps a current avoidance direction. Individual fish inherit part of this intent while retaining local variation.

Mode	Behavior
Wander	Generate slowly changing directions/targets in suitable water.
Feed	Bias the school toward a food/resource location.
Flee	Increase speed, separation, and obstacle responsiveness.
Regroup	Reduce dispersion after a panic event.
Migrate	Follow a long-range direction or environmental flow.

7. Collective obstacle response

Let leading fish detect obstacles first. Alignment, cohesion, and a school-level avoidance vector then propagate the response through the group. Optionally, the school controller can perform one larger obstacle query for the whole group and feed its result back into individual steering.

8. Local flood-fill extension

For complex cave geometry, add a bounded local voxel flood-fill as an escalation mechanism. Most obstacles can be handled by directional probes. If no candidate direction has sufficient clearance, flood-fill a small neighborhood of traversable voxels and select a short escape route around the obstacle. This is not full world pathfinding; it is a small emergency planner used only when local steering cannot resolve the situation.

9. Performance and LOD

Simulation cost should scale with relevance. Nearby fish receive full flocking and collision treatment; medium-distance fish use reduced neighbor searches; far-away groups can be represented primarily by school-level state. Rendering can still show many visual fish using instancing.

Distance / relevance	Suggested simulation
Near player	Full Boids, voxel probes, collision, animation coupling
Medium	Reduced probe count and neighbor frequency
Far	School-level movement with sparse individual state
Very far	Cheap school representation / visual-only behavior

10. Animation coupling

Visual quality depends on coupling animation to motion. Swim frequency should increase with speed; body bend and bank should respond to turning rate and lateral acceleration; orientation should smoothly follow desired velocity instead of snapping.

11. Recommended implementation sequence

1. Implement individual fish movement with acceleration and smooth orientation.
2. Add a 3D spatial hash/uniform grid for neighbor lookup.
3. Implement separation, alignment, and cohesion.
4. Add voxel occupancy/clearance queries.
5. Add directional obstacle probes and candidate-direction scoring.
6. Add temporary avoidance targets to prevent wall oscillation.
7. Introduce a school controller and behavioral modes.
8. Add predator/food interactions and panic/regroup behavior.
9. Add bounded local flood-fill for difficult voxel obstacles.
10. Add distance-based LOD and, if required, GPU/instanced rendering.

12. Core recommendation

The central design is: **Boids for social motion + a spatial grid for neighbors + voxel-aware local steering for obstacles + a school-level behavior controller + LOD for scale.** This avoids the rigidity and cost of conventional per-fish pathfinding while fitting Minecraft's voxel world particularly well.